

Contents

Framework-Walkthrough (Tag 1, 09:30–10:45)	1
Die Grundentscheidung: Live-Code statt fertiger Folien	1
Vor dem Block: Technik-Checkliste	1
Folie 1 — Einstieg (die einzige Folie vor dem Live-Teil)	2
0. Vorab: die drei Konzepte, die im Framework wichtig sind	2
1. Live-Demo zuerst (5 Min)	4
2. Das Domain-Modell — framework/arena/Models.kt (55 Zeilen)	4
3. Wie wird decide() aufgerufen? — GameEngine.step()	8
4. Absicherung gegen kaputten Bot-Code — BotExecutor (2 Min, nur konzeptuell)	9
5. Ein fertiger Beispiel-Bot — bots/examples/RandomBot.kt (29 Zeilen)	10
6. Wo trage ich meinen Code ein? — bots/teama/TeamABots.kt	10
Folie 2 — Übergang zu Arbeitsblock 2	11
Vorschlag Zeitbudget (75 Min gesamt, 09:30–10:45)	11
Was bewusst NICHT gezeigt wird (und warum)	12

Framework-Walkthrough (Tag 1, 09:30–10:45)

Begleitmaterial für den dozentengeführten Block aus [tag-1.md](#). Diese Datei ist gleichzeitig **Inhalt** (was erklärt wird) und **Regie** (wie man es präsentiert) — beides in einer Datei, damit man während des Blocks nicht zwischen zwei Dokumenten wechseln muss.

Leitsatz für den Block: Schüler schreiben ausschließlich den Body von `RobotBrain.decide()`. Alles hier Gezeigte ist fertig und wird nicht verändert — Ziel ist Verständnis, nicht Bearbeitung.

Die Grundentscheidung: Live-Code statt fertiger Folien

Für den Code-Teil werden **keine klassischen Präsentationsfolien** (PowerPoint o.ä.) empfohlen. Stattdessen: **Beamer zeigt abwechselnd Editor, Terminal und die laufende App**. Nur für Einstieg und Übergang gibt es je eine kurze Folie (siehe “Folie 1” und “Folie 2” unten).

Warum kein Foliensatz für den Code? - Schüler sehen echten, klickbaren Code — keine Screenshots, die schon beim Erstellen veraltet sein können. - Bei Fragen (“was macht `.random()` nochmal genau?”) kann man direkt im Editor nachschauen oder ausprobieren, statt an eine feste Foliensequenz gebunden zu sein. - Der Kontrast “das hier ist fertiges Framework” vs. “das hier schreibt ihr selbst” wirkt am stärksten, wenn beide Dateien als Tabs im selben Editor-Fenster offen sind und man einfach zwischen ihnen hin- und herklickt.

Was du technisch brauchst: Terminal-Fenster (für `./gradlew run`), Editor (IntelliJ oder VS Code) mit bereits geöffnetem Projekt, beide Fenster nebeneinander oder schnell per Tastenkombination wechselbar.

Vor dem Block: Technik-Checkliste

Unbedingt **vor** den Schülern durchgehen, nicht live improvisieren:

- ☐ Projekt einmal lokal bauen (`./gradlew build`), damit die Live-Demo nicht an einem kalten Gradle-Cache/Download hängen bleibt
- ☐ Editor-Schriftgröße hochstellen (mindestens 18pt, besser 20pt) — sonst ist von hinten im Raum nichts lesbar
- ☐ Terminal-Schriftgröße ebenfalls hochstellen
- ☐ Beamer-Auflösung/Spiegelung testen, bevor Schüler im Raum sind
- ☐ Diese Dateien als Tabs vorab öffnen, in dieser Reihenfolge: `Models.kt`, `Toolkit.kt`, `RandomBot.kt`, `TeamABots.kt` (bzw. die für die anwesenden Teams relevante(n) Team-Datei(en))

- docs/dozent/scrum-board.md und ausgedruckte Backlog-Karten (siehe [pdf/README.md](#)) am Board bereitlegen für den Übergang danach
-

Folie 1 — Einstieg (die einzige Folie vor dem Live-Teil)

Eine einzelne Folie oder alternativ ein Whiteboard-Anschrieb, bevor irgendetwas gestartet wird:

Titel: Framework-Walkthrough — was baut ihr heute?

Inhalt (2-3 Stichpunkte, groß geschrieben): - Sprint-Ziel: *“Bot mit mindestens zwei Verhaltensregeln, der ein Testduell übersteht.”* - Ein Satz, den man auch laut sagen sollte: *“Ihr schreibt heute eine einzige Funktion. Alles andere — Spielfeld, Regeln, Anzeige — ist schon fertig.”*

Danach die Folie weglegen/ausblenden und direkt zu Abschnitt 1 wechseln.

0. Vorab: die drei Konzepte, die im Framework wichtig sind

Bevor der eigentliche Code gezeigt wird, hier die drei Kotlin-Konzepte, die im Framework ständig vorkommen — mit einfachen Erklärungen, die man 1:1 an die Schüler weitergeben kann. Wer die Konzepte schon sicher beherrscht, kann diesen Abschnitt überspringen und direkt zu Abschnitt 1 gehen.

Regie: Dieser Abschnitt läuft weg vom Beamer — Tafel/Whiteboard oder im Stehen erklären, noch kein Code zeigen. Wenn die Klasse Interface/Enum/data class aus dem Kotlin-Vorlauf schon sicher beherrscht: Abschnitt komplett weglassen und die Zeit in Abschnitt 5/6 investieren.

Kurzform der vier Analogien, griffbereit halten: - Interface = Stellenausschreibung (“kann kochen”, aber nicht wie) - Enum = Ampel (nur Rot/Gelb/Grün, nichts dazwischen) - data class = Karteikarte mit automatischem “sind zwei Karten gleich?”-Check - sealed interface = “genau eine von mehreren fest definierten Möglichkeiten”

Was ist ein Interface?

Ein Interface ist ein **Vertrag**: es legt fest, *welche* Funktionen und Eigenschaften eine Klasse haben muss, aber nicht *wie* sie das macht.

Analogie: Ein Interface ist wie eine Stellenausschreibung. “Gesucht: jemand, der kochen kann.” Die Ausschreibung sagt nicht, *wie* gekocht wird — nur, dass die Person die Fähigkeit “kochen” mitbringen muss. Jede Person, die sich bewirbt (= jede Klasse, die das Interface implementiert), kocht vielleicht ganz anders, aber alle erfüllen den Vertrag “kann kochen”.

```
interface RobotBrain {  
    val name: String  
    fun decide(sensors: Sensors): Action  
}
```

Das bedeutet: “Jede Klasse, die RobotBrain sein will, MUSS einen name und eine Funktion decide(sensors) haben, die eine Action zurückgibt.” *Wie* die Funktion entscheidet, ist komplett offen — das ist genau der Teil, den die Schüler selbst schreiben.

Warum ist das nützlich? Die Engine (der Rest des Frameworks) muss nicht wissen, ob sie gerade mit RandomBot, ChaserBot oder dem selbstgeschriebenen Team-Bot spricht. Sie weiß nur: “Das ist ein RobotBrain, also hat es decide(), also kann ich es aufrufen.” Das ist der ganze Trick, mit dem drei komplett unterschiedliche Team-Bots im selben Spiel gegeneinander antreten können, ohne dass die Engine für jeden Bot extra angepasst werden muss.

Was ist ein Enum (enum class)?

Ein Enum ist eine **feste, abzählbare Liste von Werten**, die es geben darf — nichts anderes ist erlaubt.

Analogie: Eine Ampel kann nur Rot, Gelb oder Grün sein. Es gibt kein “ein bisschen Gelb” oder “Blau”. Ein Enum in Kotlin bildet genau so eine feste Auswahl ab.

```
enum class Direction(val dx: Int, val dy: Int) {  
    NORTH(0, -1), EAST(1, 0), SOUTH(0, 1), WEST(-1, 0)  
}
```

Hier gibt es genau vier mögliche Richtungen — `Direction.NORTH`, `.EAST`, `.SOUTH`, `.WEST`. Man kann nicht aus Versehen `Direction.NORDWEST` schreiben, das würde gar nicht kompilieren. Das schützt vor Tippfehlern und ungültigen Werten.

Das `(val dx: Int, val dy: Int)` dahinter ist etwas Zusätzliches: **jeder** Wert des Enums trägt selbst noch zwei Zahlen mit sich herum. `dx/dy` beschreiben, wie sich die x/y-Koordinate verändert, wenn man einen Schritt in diese Richtung geht. Also: `Direction.NORTH.dx` ist 0, `Direction.NORTH.dy` ist -1. Man kann sich das wie ein Enum mit eingebauten “Zusatzinfos” pro Wert vorstellen.

Stolperfalle, die man den Schülern vorab sagen sollte: In diesem Framework wächst die y-Koordinate nach unten (wie bei Bildschirm-Koordinaten, nicht wie im Matheunterricht). `NORTH` (nach oben auf dem Bildschirm) bedeutet deshalb `y - 1`, nicht `y + 1`. Das verwirrt fast immer beim ersten Mal.

Was ist eine data class?

Eine `data class` ist eine Klasse, die nur Daten zusammenhält (kein eigenes Verhalten), und für die Kotlin automatisch nützliche Funktionen mitliefert: Vergleichen zweier Objekte (`==`), eine lesbare Text-Darstellung, und eine Kopier-Funktion.

```
data class Position(val x: Int, val y: Int)
```

Das ist im Grunde nur “eine Position hat ein x und ein y”. Ohne `data` müsste man selbst Code schreiben, damit zwei `Position`-Objekte mit gleichen Werten auch als “gleich” erkannt werden. Mit `data` passiert das automatisch.

Was ist ein sealed interface?

Das ist die Steigerung von Interface + Enum: ein `sealed interface` legt fest, **welche festen, unterschiedlich aufgebauten Möglichkeiten** es für etwas gibt — anders als ein Enum, wo jeder Wert gleich aussieht (nur Name + evtl. Zusatzwerte), können bei einem `sealed interface` die einzelnen Möglichkeiten ganz unterschiedliche Daten enthalten.

```
sealed interface Action {  
    data class Move(val direction: Direction) : Action  
    data class Shoot(val direction: Direction) : Action  
    data object Wait : Action  
}
```

Eine `Action` ist **immer genau eine** von drei Dingen: - `Action.Move(direction)` — enthält eine Richtung, in die man sich bewegen will - `Action.Shoot(direction)` — enthält eine Richtung, in die man schießen will - `Action.Wait` — enthält gar nichts, einfach “nichts tun”

Der Unterschied zum Enum: `Move` und `Shoot` tragen zusätzlich eine `Direction` mit sich, `Wait` trägt gar nichts. Das wäre mit einem einfachen Enum nicht abbildbar.

Für die Schüler reicht: *“Ihr müsst aus eurer decide()-Funktion immer genau eines von diesen drei Dingen zurückgeben: Action.Move(Direction.NORTH), Action.Shoot(Direction.EAST), oder Action.Wait.”* Wie man selbst einen sealed interface definiert, ist **nicht** Lernziel — nur die Benutzung.

1. Live-Demo zuerst (5 Min)

Regie — Wechsel zu: Terminal, dann App-Fenster.

1. Terminal einblenden, `./gradlew run` eintippen und ausführen (dauert je nach Rechner 10-30 Sekunden — diese Zeit für den Sprechtext von Folie 1 nutzen, nicht schweigend warten).
2. Sobald die App startet: RandomBot und ChaserBot als Teilnehmer auswählen, Match starten.
3. App-Fenster maximieren, damit alle im Raum das Raster sehen.

Sprechzeile: *“Schaut euch das an — zwei fertige Bots, die gegeneinander kämpfen. Das ist keine Zauberei, das ist Code, den wir uns jetzt gemeinsam anschauen. Am Ende des heutigen Tages soll euer eigener Bot hier genauso mitspielen können.”*

Kurz auf UI-Elemente zeigen (nicht erklären, nur benennen): Arena links (10×10-Raster), Scoreboard und Log rechts — da sehen Schüler später, was ihr Bot in jedem Moment tut.

2. Das Domain-Modell — framework/arena/Models.kt (55 Zeilen)

Regie — Wechsel zu: Editor, Tab Models.kt. Von oben nach unten scrollen, bei jedem Typ kurz stehen bleiben, Reihenfolge exakt wie unten: Direction -> Position -> RobotState -> Sensors -> Action -> RobotBrain -> Toolkit.kt.

Konkrete Methode, die Aufmerksamkeit hochhält: Bei jedem neuen Typ erst laut die Frage stellen *“Was glaubt ihr, wofür ist das da?”*, 2-3 Antworten aus dem Raum sammeln, dann erst die eigene Erklärung geben.

Bei jedem Typ zusätzlich ein Nutzungsbeispiel zeigen, nicht nur die Definition — die reine Typ-Definition allein bleibt abstrakt, das Beispiel macht den Bezug zur eigenen Aufgabe sichtbar. Am besten direkt in einer Scratch-Datei oder als Kommentar im Editor eintippen und laufen lassen.

Direction

```
enum class Direction(val dx: Int, val dy: Int) {  
    NORTH(0, -1), EAST(1, 0), SOUTH(0, 1), WEST(-1, 0)  
}
```

Siehe Erklärung “Was ist ein Enum” oben. Konkret zeigen: Direction.NORTH.dx ist 0, Direction.NORTH.dy ist -1 — Richtung nach oben auf dem Bildschirm.

Position

```
data class Position(val x: Int, val y: Int) {  
    fun moved(direction: Direction) = Position(x + direction.dx, y + direction.dy)  
}
```

Konzept dahinter: Eine Position ist einfach ein Koordinatenpaar auf dem 10×10-Raster. (0, 0) ist oben links (typisch für Computergrafik, anders als im Matheunterricht üblich, wo (0,0) oft in der Mitte oder unten links liegt).

Warum ist das eine eigene Klasse und nicht einfach zwei lose Zahlen x und y? Weil man dann nie zwei zusammengehörige Werte getrennt herumreichen muss ("wo ist eigentlich das y zu diesem x?") und weil Kotlin für `data class` automatisch Vergleiche mitliefert: zwei `Position`-Objekte mit gleichem x und y gelten mit `==` automatisch als gleich. Das wird später wichtig, z.B. um zu prüfen "steht der Gegner auf derselben Position wie ich?"

Die Funktion `moved(direction)` ist die praktische Anwendung des Enums von oben: "Gib mir die Position, die einen Schritt in diese Richtung liegt."

Regie — besonders hervorheben mit einem Beispiel an der Tafel: Roboter steht auf (3, 3), bewegt sich NORTH -> neue Position ist $(3 + 0, 3 + (-1)) = (3, 2)$. Bewusst durchrechnen — abstrakte Formeln bleiben bei 10.-Klässlern schlechter hängen als ein durchgerechnetes Zahlenbeispiel.

Kurzer Hinweis zur Kurzschreibweise `fun moved(...) = Position(...)`: das ist nur eine kürzere Schreibweise für eine Funktion, die nur eine einzige Berechnung zurückgibt — entspricht `fun moved(...): Position { return Position(...) }`.

Konkretes Nutzungsbeispiel aus `decide()`: Angenommen, ein Bot will wissen, ob er sich gerade am linken Rand befindet:

```
val amLinkenRand = sensors.self.position.x == 0
```

Oder: die Distanz zu einem Gegner grob abschätzen (nur x-Differenz, ohne Wurzelziehen — für die Bot-Logik reicht das):

```
val gegner = sensors.others.first()
val distanzX = sensors.self.position.x - gegner.position.x
```

`sensors.self.position` ist der Einstiegspunkt für praktisch jede räumliche Entscheidung, die ein Bot trifft — Distanz zu Gegnern, Nähe zum Rand, Richtung zum Zentrum, etc. Diese Zeile lohnt sich, laut mitzusprechen: *"Jede Positions-Frage, die euer Bot beantworten will, fängt mit `sensors.self.position` an."*

RobotState

```
data class RobotState(
    val id: String,
    val teamName: String,
    val position: Position,
    val health: Int
) {
    val alive: Boolean get() = health > 0
}
```

Konzept dahinter: Das ist der komplette "Steckbrief" eines Roboters zu einem bestimmten Zeitpunkt: eine ID (z.B. "bot-0", wird von der Engine automatisch vergeben), ein Name, eine Position, und die aktuelle Gesundheit (Health Points, HP).

Warum ein `RobotState` und nicht einfach vier einzelne Werte? Weil man dann alles, was zu einem Roboter gehört, mit einer Variable herumreichen kann. `Sensors` enthält z.B. eine ganze `List<RobotState>` — ohne dieses Bündel müsste man vier separate Listen (eine für IDs, eine für Positionen, eine für Health, ...) synchron halten, was extrem fehleranfällig wäre.

alive genauer erklärt: Es steht **kein fester Wert** dahinter, sondern eine kleine Berechnung (`get()` = ...). Jedes Mal, wenn man `robotState.alive` abfragt, wird live geprüft: "ist health größer als 0?" Der Vorteil: man muss sich nie sorgen, ob `alive` "vergessen" wurde zu aktualisieren, wenn `health` sich ändert — es kann gar nicht aus dem Takt geraten, weil es nicht separat gespeichert wird, sondern jedes Mal frisch berechnet wird.

Konkretes Nutzungsbeispiel aus `decide()`: Die eigene Gesundheit abfragen, um zwischen Angriff und Flucht zu entscheiden (das ist praktisch Story 1.3 aus dem Backlog):

```
if (sensors.self.health < 20) {  
    // fliehen  
} else {  
    // normal weiterkämpfen  
}
```

Oder: nur noch lebende Gegner betrachten, die eine bestimmte Bedingung erfüllen, z.B. den mit der niedrigsten Gesundheit als Ziel wählen (Story 3.2):

```
val schwachsterGegner = sensors.others.minByOrNull { it.health }
```

`sensors.self` ist immer ein `RobotState` — der eigene. Jeder Eintrag in `sensors.others` ist ebenfalls ein `RobotState` — aber von einem Gegner. Das ist derselbe Bauplan, nur für unterschiedliche Roboter befüllt.

Sensors

```
data class Sensors(  
    val self: RobotState,  
    val others: List<RobotState>,  
    val arenaWidth: Int,  
    val arenaHeight: Int,  
    val tick: Int  
)
```

Konzept dahinter: `Sensors` ist **das einzige Fenster zur Welt**, das ein Bot hat. Es wird von der Engine **jeden Tick neu gebaut** und an `decide()` übergeben — der Bot bekommt also bei jedem Aufruf einen druckfrischen Schnappschuss des aktuellen Spielstands, nie einen veralteten.

Analogie: Stellt euch vor, ihr spielt Schach, aber mit verbundenen Augen — jemand sagt euch einmal pro Zug genau, wo alle Figuren gerade stehen, und dann müsst ihr sofort entscheiden. Ihr könnt euch nicht “vorher schon gemerkt haben, wo der Gegner letzten Zug war” — es zählt nur, was gerade in `Sensors` drinsteht.

Enthält genau fünf Dinge — wichtig, das explizit durchzugehen, das ist buchstäblich **alles**, was ein Bot wissen darf:

- `self` — der eigene `RobotState` (eigene Position, eigene Health, eigene ID)
- `others` — eine `List<RobotState>` aller **noch lebenden** anderen Roboter. Tote Roboter tauchen hier nicht mehr auf — ein Bot muss also nie selbst prüfen, ob ein Gegner schon tot ist, das hat die Engine schon für ihn erledigt.
- `arenaWidth/arenaHeight` — Größe des Spielfelds (beides 10, aber bewusst nicht fest im Code der Bots verdrahtet, sondern übergeben — falls sich die Arena-Größe mal ändert, muss kein Bot-Code angepasst werden)
- `tick` — die aktuelle Runden-Nummer, falls ein Bot zeitabhängiges Verhalten will (z.B. “in den ersten 10 Ticks patrouillieren, danach angreifen”)

Warum so eingeschränkt — kein Zugriff auf die Engine selbst, keine Erinnerung an vorherige Ticks? Zwei Gründe, die man den Schülern nennen kann: 1. **Fairness:** Kein Bot kann “schummeln”, indem er auf interne Engine-Daten zugreift oder Dinge sieht, die er im echten Spiel nicht sehen dürfte. 2. **Einfachheit:** `decide()` muss nur eine Frage beantworten — “was tue ich JETZT, basierend auf dem, was ich JETZT sehe?” Kein Bot muss sich selbst Zustand über mehrere Ticks hinweg merken, es sei denn er will es (das ist optional möglich über eine eigene `var` in der Bot-Klasse, siehe Story 3.1 und 3.5 im Backlog).

Konkretes Nutzungsbeispiel — ein kompletter Mini-Bot, der alle vier Sensor-Felder mindestens einmal benutzt:

```
override fun decide(sensors: Sensors): Action {
    // Gibt es überhaupt noch einen Gegner?
    val gegner = sensors.others.firstOrNull() ?: return Action.Wait

    // Stehe ich mit dem Gegner in derselben Zeile (gleiches y)?
    return if (sensors.self.position.y == gegner.position.y) {
        Action.Shoot(Direction.EAST)
    } else {
        Action.Move(Direction.SOUTH)
    }
}
```

Diese vier Zeilen sind fast eine Vorschau auf Story 2.2 aus dem Backlog (Zielen auf Gegner in Sichtlinie) — gut geeignet, um zu zeigen, dass die Storys später genau auf diesen Bausteinen aufbauen.

Action

```
sealed interface Action {
    data class Move(val direction: Direction) : Action
    data class Shoot(val direction: Direction) : Action
    data object Wait : Action
}
```

Konzept dahinter: Siehe ausführliche Erklärung “Was ist ein sealed interface” oben — hier nochmal konkret auf diesen Anwendungsfall bezogen. Action ist die **einzigste Art, wie ein Bot mit der Spielwelt kommunizieren kann**. Ein Bot kann nichts direkt verändern (nicht selbst seine position setzen, nicht selbst health abziehen) — er kann nur eine Action *vorschlagen*, und die Engine setzt sie dann nach ihren eigenen Regeln um (inklusive der Konfliktregeln aus Abschnitt 3, z.B. wenn zwei Bots ins selbe Feld wollen).

Das ist ein wichtiges Prinzip, das sich lohnt laut auszusprechen: *“Euer Bot schlägt vor, die Engine entscheidet.”* Ein Bot kann z.B. Action.Move(Direction.NORTH) zurückgeben, obwohl direkt nördlich eine Wand oder ein anderer Roboter ist — die Engine prüft das und lässt die Bewegung im Zweifel einfach ins Leere laufen. Der Bot bekommt dafür keine Fehlermeldung, es passiert einfach nichts.

Warum drei Möglichkeiten und nicht z.B. drei einzelne Funktionen move(), shoot(), wait()? Weil decide() **genau eine** Aktion pro Tick zurückgeben muss — nie zwei gleichzeitig (nicht “bewege dich UND schieße”). Der sealed interface-Typ erzwingt das automatisch: die Funktion hat den Rückgabetypp Action, und Action kann eben immer nur eines der drei Dinge gleichzeitig sein.

Konkrete Nutzungsbeispiele, alle drei Fälle:

```
Action.Move(Direction.NORTH)    // einen Schritt nach oben gehen
Action.Shoot(Direction.WEST)    // nach links schießen
Action.Wait                     // nichts tun (kein Klammern, da kein Parameter)
```

Regie — bei Action besonders auf die Klammer-Fälle hinweisen: Action.Wait wird **ohne** Klammern geschrieben (Action.Wait, nicht Action.Wait()), weil es ein data object ist — ein Singleton, kein Konstruktor-Aufruf. Move und Shoot dagegen brauchen Klammern mit einer Direction drin, weil es data classes sind, die einen Wert mitbekommen. Dieser Unterschied verwirrt fast immer beim ersten eigenen Schreiben, wenn er nicht vorher explizit erwähnt wurde — kurz explizit erwähnen.

RobotBrain

```
interface RobotBrain {  
    val name: String  
    fun decide(sensors: Sensors): Action  
}
```

Siehe ausführliche Erklärung “Was ist ein Interface” oben. Das ist der Vertrag, den jeder Bot erfüllen muss. **Das ist die einzige Sache, die Schüler wirklich selbst bauen: eine Klasse, die diesen Vertrag erfüllt.**

Zusammenfassender Satz für die Tafel: *“Ein Bot ist nichts anderes als ein Name plus eine Funktion, die aus Sensors eine Action macht.”*

Toolkit.kt — was euch die Bibliothek abnimmt

Regie: Datei im Editor kurz aufklappen, nicht Zeile für Zeile lesen — nur zeigen, dass sie existiert und im selben Package wie Models.kt liegt. Kurzer Ausblick, kein Detail-Deep-Dive (2-3 Min).

Neben Models.kt gibt es im selben Package (framework/arena) noch Toolkit.kt — fertige Funktionen für genau die Rasterrechnung, die sonst von Hand gemacht werden müsste (Abstand zwischen zwei Positionen, nächster/schwächster Gegner, Richtung zu einem Ziel, Sichtlinien-Check, Rand/Mitte-Erkennung).

```
val ziel = sensors.nearestEnemy() ?: return Action.Wait  
if (sensors.canShoot(ziel)) {  
    return Action.Shoot(sensors.self.position.directionTo(ziel.position)!!)  
}
```

Sprechzeile: *“Distanz, Richtung, nächster Gegner, Sichtlinie — das rechnet diese Datei für euch aus. Ihr ruft nur die Funktion auf. Eure Aufgabe ist es, diese fertigen Bausteine sinnvoll zu kombinieren, nicht sie neu zu erfinden.”* Weil die Funktionen im selben Package wie Models.kt liegen, reicht der gewohnte `import framework.arena.*` — kein zusätzlicher Import-Pfad.

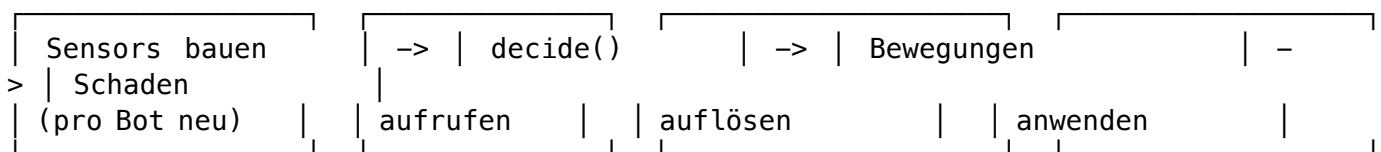
Vollständige Funktionsliste zum Nachschlagen für später: [docs/toolkit-referenz.md](#) — das ist die Datei, auf die man Schüler verweist, wenn während der Arbeitsblöcke die Frage kommt “wie berechne ich X?”.

3. Wie wird decide() aufgerufen? — GameEngine.step()

Regie — Wechsel zu: weg vom Beamer, hin zur Tafel/zum Whiteboard. Das ist bewusst ein Medienwechsel — signalisiert “jetzt nicht mehr Kotlin-Syntax, jetzt der große Ablauf”. Die Datei GameEngine.kt selbst ist mit über 300 Zeilen zu dicht und nutzt fortgeschrittene Kotlin-Konzepte (Higher-Order- Functions, groupBy, Destructuring), die für Kotlin-Anfänger noch zu viel wären. Stattdessen den Ablauf **als Diagramm an die Tafel malen:**

Sensors bauen -> decide() aufrufen -> Bewegungen auflösen -> Schaden anwenden

Ausführlicher als Kasten-Diagramm:



Erklärung Schritt für Schritt (als Sprechtext, nicht an die Tafel schreiben):

1. **Für jeden lebenden Bot** wird ein frisches Sensors-Objekt gebaut — mit dem aktuellen Stand von allem.
2. `decide()` wird **einmal pro Bot pro Tick** aufgerufen. Nie öfter, nie für zwei Bots gleichzeitig vermischt.
3. **Alle** Antworten (Move/Shoot/Wait) von **allen** Bots werden erst gesammelt, bevor überhaupt etwas passiert.
4. Erst danach werden alle Bewegungen gleichzeitig aufgelöst, dann alle Schüsse gleichzeitig ausgewertet.

Wichtig zu betonen: Ein “Tick” ist eine Zeiteinheit im Spiel, in der *alle* Bots gleichzeitig einmal entscheiden dürfen — wie eine Runde bei einem Brettspiel, nur dass alle Spieler gleichzeitig ziehen statt nacheinander.

Zwei Aha-Momente, die man als Geschichte erzählen sollte (nicht als Regel)

Diese zwei Dinge erklären scheinbar “komische” Spielsituationen, die Schüler später beim Testen beobachten werden. **Regie:** bewusst als kleine Geschichte erzählen, nicht an die Tafel schreiben — es sind spätere Aha-Momente, keine Merksätze zum Auswendiglernen.

Geschichte 1 — der Bewegungs-Konflikt: *“Stellt euch vor, zwei Bots stehen nebeneinander und wollen beide ins selbe freie Feld ziehen. Was passiert? Antwort: keiner von beiden bewegt sich in diesem Tick. Das Spiel entscheidet nicht per Zufall, wer zuerst darf — es lässt einfach beide stehen. Wenn ihr also später seht, dass euer Bot plötzlich ‘hängen bleibt’, obwohl das Feld vor ihm frei aussah: vielleicht wollte ein Gegner im selben Moment auf dasselbe Feld.”*

Geschichte 2 — der gesammelte Schaden: *“Alle Schüsse in einem Tick werden gesammelt und dann gemeinsam ausgewertet — nicht einer nach dem anderen. Das heißt: Wenn zwei Bots im selben Tick auf denselben dritten Bot schießen, und der dadurch stirbt, dann zählen trotzdem **beide** Schüsse noch, egal wer ‘zuerst’ geschossen hätte. Es gibt kein ‘zu spät, der ist schon tot’ innerhalb desselben Ticks.”*

Warum das so gebaut ist (nur bei Nachfrage erklären, nicht von selbst vertiefen): Sonst würde das Ergebnis eines Spiels davon abhängen, in welcher Reihenfolge die Engine zufällig die Bots intern abarbeitet — das wäre unfair und nicht reproduzierbar.

4. Absicherung gegen kaputten Bot-Code — BotExecutor (2 Min, nur konzeptuell)

Kein Code zeigen, nur die Kernaussage — das nimmt den Schülern die Angst, beim Ausprobieren “das ganze Spiel kaputt zu machen”:

“Wenn euer Bot-Code einen Fehler hat — eine Endlosschleife, eine Exception, egal was — dann crasht nicht das ganze Programm. Euer Bot macht in diesem Fall einfach gar nichts (Wait), bis ihr den Fehler behoben habt. Ihr könnt also mutig ausprobieren, ohne dass ihr den anderen Teams das Spiel kaputt macht.”

Falls jemand nachfragt, wie das technisch geht: *“Jeder Bot-Aufruf läuft in einem eigenen kurzen Zeitfenster. Wenn er dieses Fenster verpasst oder abstürzt, wird das abgefangen.”* Mehr Tiefe ist für den Rahmen nicht nötig.

Falls jemand fragt, warum sich sein Bot gelegentlich auch ohne Fehler mal “komisch” bewegt: *“Alle 25 Ticks erzwingt das Spiel bei jedem Bot einen zufälligen Schritt. Das verhindert, dass zwei Bots sich für immer gegenseitig blockieren, wenn sie z.B. beide aufs gleiche Feld wollen. Das ist normales Verhalten, kein Fehler in eurem Code.”*

5. Ein fertiger Beispiel-Bot — bots/examples/RandomBot.kt (29 Zeilen)

Regie — Wechsel zu: Editor, Tab RandomBot.kt. Das ist der wichtigste Programmpunkt des ganzen Blocks — hier sehen Schüler zum ersten Mal, wie ein RobotBrain tatsächlich implementiert aussieht.

Konkrete Methode: Vor jeder Zeile fragen “Was glaubt ihr, macht diese Zeile?” und 10-15 Sekunden Stille aushalten, bevor man selbst erklärt — das ist unbequem, aber aktiviert deutlich mehr als reines Vorlesen.

```
class RandomBot(override val name: String = "RandomBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val direction = Direction.entries.random()
        return if (Random.nextBoolean()) Action.Move(direction) else Action.Shoot(direction)
    }
}
```

Zeile für Zeile:

- `class RandomBot(...) : RobotBrain` — das `: RobotBrain` heißt “diese Klasse erfüllt den RobotBrain-Vertrag von oben”. Wenn hier `name` oder `decide()` fehlen würden, würde das Programm gar nicht kompilieren — Kotlin zwingt einen, den Vertrag vollständig zu erfüllen.
- `override val name: String = "RandomBot"` — `override` heißt “ich erfülle hiermit das, was das Interface von mir verlangt hat”. Der Bot heißt standardmäßig “RandomBot”.
- `Direction.entries` — das ist die Liste aller vier Enum-Werte ([NORTH, EAST, SOUTH, WEST]). `.random()` pickt zufällig einen davon aus.
- `if (Random.nextBoolean()) Action.Move(direction) else Action.Shoot(direction)` — eine Münzwurf-Entscheidung: 50% Bewegen, 50% Schießen, jeweils in die zufällig gewählte Richtung.

Wichtiger Kotlin-Hinweis: `if/else` wird hier nicht als reine Verzweigung benutzt (wie “wenn X, mach A, sonst mach B”), sondern gibt selbst einen **Wert** zurück, der direkt hinter `return` steht. Das ist in Kotlin normal, aber für Einsteiger aus anderen Sprachen oft neu — kurz erwähnen.

Optional, nur wenn Zeit bleibt: `ChaserBot.kt` zeigen (sucht nächstgelegenen Gegner über Toolkit-Funktionen). Das ist aber schon Story-2.3-Niveau — nicht erzwingen, wenn die Zeit knapp wird.

6. Wo trage ich meinen Code ein? — bots/teama/TeamABots.kt

Regie — Wechsel zu: Editor, Tab TeamABots.kt (bzw. passende Team-Datei). Das eigentliche Ziel des ganzen Blocks: jeder weiß danach genau, wo er/sie schreibt.

```
package bots.teama
```

```
import framework.arena.Action
import framework.arena.Direction
import framework.arena.RobotBrain
import framework.arena.Sensors
```

```
class MeinBot(override val name: String = "Team A - MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        // TODO: Bewege dich stattdessen in Richtung des nächsten Gegners
        // TODO: Schieße wenn ein Gegner in Sichtlinie ist
        // TODO: Reagiere auf niedrige eigene HP (sensors.self.health), z.B. mit Flucht
        return Action.Move(Direction.SOUTH)
    }
}
```

```
}
```

```
val teamABots: List<RobotBrain> = listOf(MeinBot())
```

Live demonstrieren: Action.Move(Direction.SOUTH) zu Action.Move(Direction.entries.random()) ändern, zurück zum Terminal wechseln, ./gradlew run neu starten, zeigen dass der Bot jetzt zufällig läuft.

Sprechzeile: *“Das war’s schon — eine Zeile geändert, neu gestartet, der Bot verhält sich anders. Genau das macht ihr jetzt gleich selbst, nur mit eurer eigenen Logik.”*

Der mit Abstand häufigste Compile-Fehler, den man vorab ankündigen sollte: eine neue Bot-Klasse wird angelegt, aber vergessen, sie unten in die teamXBots-Liste einzutragen (listOf(MeinBot())) — der Bot compiliert dann zwar, taucht aber nicht in der App-Auswahl auf.

Sprechzeile dazu: *“Wenn ihr eine ganz neue Bot-Klasse anlegt: vergesst nicht, sie unten in die Liste einzutragen — sonst taucht sie nicht in der Bot-Auswahl auf. Das ist der Fehler, der euch als Erstes passieren wird, und das ist völlig normal.”* Explizit vorab sagen, damit Schüler das beim ersten Mal selbst erkennen statt zu verzweifeln.

Folie 2 — Übergang zu Arbeitsblock 2

Titel: Jetzt seid ihr dran

Inhalt: - Story 1.1 (Zufällige Bewegung) als Einstieg - Erinnerung: Pair-Programming, Driver/Navigator wechseln sich ab - Erinnerung: neue Klasse -> in teamXBots-Liste eintragen

Danach direkt zu den Rechnern schicken, Board zeigen (Karten liegen schon in der Backlog-Spalte, siehe scrum-board.md).

Vorschlag Zeitbudget (75 Min gesamt, 09:30–10:45)

Zeit	Dauer	Inhalt
09:30–09:35	5 Min	Live-Demo (RandomBot vs. ChaserBot)
09:35–10:00	25 Min	Models.kt komplett durchgehen, inkl. Beispiele je Typ + kurzer Toolkit.kt-Ausblick (Abschnitt 2)
10:00–10:08	8 Min	GameEngine.step() als Tafel-Diagramm (Abschnitt 3)
10:08–10:11	3 Min	BotExecutor-Sicherheitsnetz, nur Kernaussage (Abschnitt 4)
10:11–10:28	17 Min	RandomBot.kt gemeinsam Zeile für Zeile (Abschnitt 5)
10:28–10:40	12 Min	TeamXBots.kt — wo trage ich ein, Live-Beispiel bauen und laufen lassen (Abschnitt 6)
10:40–10:45	5 Min	Fragen, Übergang zu Arbeitsblock 2 (Story 1.1)

Die Grundkonzepte (Abschnitt 0: Interface, Enum, data class, sealed interface) sind hier bewusst nicht mehr im Standardbudget — sie wurden durch das neue 09:15–09:30-Kurz-Intro zu Git/Gradle verdrängt. Wenn die Klasse sie aus dem Kotlin-Vorlauf noch nicht sicher beherrscht, kurz in Abschnitt 2 nebenbei miterklären (z.B. bei `Direction` das Enum-Konzept, bei `Action` den sealed interface), statt einen eigenen Block dafür zu reservieren.

Was bewusst NICHT gezeigt wird (und warum)

- **BotExecutor, resolveMoves()/resolveShots(), App.kt im Detail** — diese Dateien nutzen fortgeschrittene Kotlin-Features, die für einen 90-Minuten-Einstieg zu viel wären: Java-Interop mit Threads (`BotExecutor`), Higher-Order-Functions wie `groupBy/mapNotNull` (`GameEngine`), und Compose-spezifische Konzepte wie `remember/LaunchedEffect` (`App.kt`). Bei Nachfragen reicht: *“Das ist fertiges Framework, das müsst ihr nie anfassen — schaut’s euch gerne selbst an, wenn ihr neugierig seid, aber es ist nicht Teil der Aufgabe.”*
- **Wie man selbst ein Interface, einen Enum oder einen sealed interface definiert** — Lernziel ist nur, diese Konzepte zu *erkennen* und zu *benutzen* (z.B. `Action.Move(Direction.NORTH)` schreiben), nicht selbst neue davon zu bauen.